```
      val shardRegion: ActorRef[ShardingEnvelope[Container.Command]] =
        sharding.init(entityDefinition)


      val containerId = "id-1"
      val cargo = Container.Cargo("id-c","sack",3)

      shardRegion ! ShardingEnvelope(
        containerId,
        Container.AddCargo(cargo))

      val probe = createTestProbe[List[Container.Cargo]]()

      val container: EntityRef[Container.Command] =                ❷
        sharding.entityRefFor(Container.TypeKey, containerId)

      container ! Container.GetCargos(probe.ref)                   ❸
      probe.expectMessage(List(cargo))
    }
  }
}
```

❶ Only shows the import-related cluster sharding

❷ Gets EntityRef by containerId

❸ Sends a message without ShardingEnvelope

As you can see, working with cluster sharding is simple. But keep in mind that sharding is a cluster feature, so you need to provide a configuration for it. The following configuration makes the preceding test work.

Listing 9.3 Container configuration

```
akka {
  actor {
    provider = cluster

    serialization-bindings {
      "example.persistence.CborSerializable" = jackson-cbor      ❶
    }
  }

  remote {
    artery {                                                     ❷
      canonical.hostname = 127.0.0.1
      canonical.port = 2552
    }
  }

  cluster {
    seed-nodes = [
      "akka://ContainerSpec@127.0.0.1:2552"]                     ❸
  }
}
```

❶ Maps CborSerializable to jackson-cbor serialization

❷ Sets the hostname and port

❸ Includes the name of the Spec as the ActorSystem in the URI

Because `ScalaTestWithActorTestKit` is used in the test, the name of the `ActorSystem` is the name of the `Spec`: `ContainerSpec`. Therefore, it is set as such in